

Training & Certificate Tracker

Realisatiedocument

Mauro Leonardo
Student Bachelor in de Toegepaste Informatica – Applicatieontwikkeling

Inhoudsopgave

1. INLEIDING	4
2. ANALYSE	3
2.1. Frontend-analyse	3
2.1.1. Scoring	3
2.1.2. Conclusie	4
2.2. Backend-analyse	4
2.3. Projectmanagementtool	5
2.4. Besluit	5
3. BACKEND DEVELOPMENT	6
3.1. Domain-laag	6
3.2. Applicatie-laag met MediatR	7
3.3. Opslaan van data	7
3.4. Security	8
3.5. Orkestratie	8
3.6. Uitdagingen	9
4. FRONTEND DEVELOPMENT	10
4.1. Architectuur	10
4.1.1. De voordelen van componenten	10
4.1.2. Services & facade-pattern	10
4.1.3. Entiteiten als frontend models	11
4.2. Belangrijke pagina's	11
4.2.1. Data Tables	12
4.2.2. Team skill matrix	12
4.2.3. Dashboard	12
5. PERSOONLIJKE GROEI	13
6. BESLUIT	14
LITERATUURLIJST	15
BIJLAGEN	16

Lijst van figuren

Figuur 1: Clean Architecture lay-out van de backend	16
Figuur 2: Test folders voor architectuur, domain en integratie testen	16
Figuur 3: Sealed User class met zijn eigenschappen	17
Figuur 4: Create methode voor gebruiker die een correcte instantie teruggeeft	17
Figuur 5: Email Value Object met eigen validatie	18
Figuur 6: Result class voor succes en mislukt	18
Figuur 7: Create training endpoint die mediatr aanspreekt	19
Figuur 8: Create training command, handler en validatie	19
Figuur 9: Linq query om een training op te halen via een id, in de mediatr handler	20
Figuur 10: EF-interceptor voor het soft deleten van een record	20
Figuur 11: Training repository en de create methode voor een training	21
Figuur 12: De interface die geïmplementeerd wordt door de training repository	21
Figuur 13: De Aspire AppHost die de projecten, database en blob storage start	22
Figuur 14: Accountmanagement met de herbruikbare dropdown component open	22
Figuur 15: Certificaat type management met de herbruikbare dropdown component open	23
Figuur 16: De API-service voor certificaten die de API-calls bevat	23
Figuur 17: De facade voor certificaten die de API-service gebruikt, met extra functionaliteiten zoals loading	24
Figuur 18: Frontend modellen voor certificaten als interfaces	24
Figuur 19: Accountmanagementpagina als data table	25
Figuur 20: Certificate portfolio pagina als data table, met update venster open	25
Figuur 21: Teammanagementpagina als data table, met create venster open met validatiefoutenvenster	26
Figuur 22: team skill matrix met muis over een record	26
Figuur 23: team skill matrix met muis over een gebruikersnaam	27
Figuur 24: Dashboard pagina als administrator	27

1. Inleiding

AllPhi is een IT-Consultancybedrijf met een hoofdkantoor in Westerlo, en kantoren in Brussel en Gent. Ze zijn gespecialiseerd in Microsoft-technologieën, zoals .NET, Azure en AI. De wereld van IT is er een die constant evolueert. Hierdoor moeten consultants trainingen volgen om mee te blijven met de evolutie en certificaten te behalen om te bewijzen dat ze een skill onder de knie hebben. Het bijhouden van de trainingen en certificaten gebeurt momenteel door overhead, waardoor consultants zelf ook een overzicht moeten bijhouden voor zichzelf.

De 'Training & Certificate Tracker' is een interne applicatie die het bijhouden van trainingen en certificaten centraliseert en digitaliseert. Hierdoor kunnen zowel de consultants als overhead snel de records bekijken en een idee krijgen van het portfolio van een consultant. Consultants loggen zelf hun trainingen en behaalde certificaten, kunnen ze snel terugvinden en zo makkelijk een portfolio opstellen van hun skills. Overhead gebruikt deze data om de perfecte consultant te kiezen voor de opdrachten van de klanten, en te bekijken waar consultants eventueel kunnen bijscholen of certificaten kunnen behalen.

In dit document worden de gekozen technologieën die gebruikt zijn tijdens de realisatie van deze applicatie besproken. De keuzes zijn vooraf gemaakt door de technologie manager van AllPhi, maar er is kort gezocht naar eventuele alternatieven om een tegenvoorstel te doen.

Dit document is ingedeeld in de analyse, de backend, de frontend, en de persoonlijke groei tijdens de stage. Het document wordt afgesloten met een besluit. De frontend gaat over hoe de applicatie er visueel uitziet, de gebruikerservaring en hoe dit technisch opgebouwd is. De backend gaat over hoe de benodigdheden voor de applicatie technisch gerealiseerd zijn. Beide bespreken ook kort welke technische keuzes er gemaakt zijn tijdens het ontwikkelen. De persoonlijke groei gaat over welke persoonlijke skills of ervaringen er bijgeleerd zijn en hoe er persoonlijk gegroeid is tijdens de stage. Het besluit vat kort de belangrijkste zaken van de stage samen en evalueert kort hoe ze verlopen is.

De stage is heel individueel, wat wil zeggen dat er geen stagementor is die constant monitort. Dit is bewust gedaan, omdat individueel werken en keuzes maken een belangrijke skill is voor een consultant. Natuurlijk hoort feedback en met voorstellen komen hier ook bij, wat de manier is om tijdens het ontwikkelen onzekerheden weg te werken, of om een denkwijze te controleren. Het is de bedoeling dat er geen vragen gesteld worden, maar met een idee naar een consultant te gaan en na te vragen of de denkwijze logisch is.

2. Analyse

Tijdens de analyse wordt er besproken welke technologieën gebruikt zijn tijdens de stage, welke alternatieven er waren, en waarom een specifieke technologie gebruikt is. Een analyse werd gemaakt voor de start van de stage door de Technology Manager van AllPhi, maar keuzes die hierin beschreven zijn, zijn tijdelijk in vraag gesteld om eventueel relevantere alternatieven te vinden, of om erachter te komen waarom deze keuzes gemaakt zijn. Ook al stonden de high-level keuzes al vast, was het nuttig om verschillende opties te vergelijken om bijvoorbeeld makkelijker een betere keuze te maken tijdens een volgend project.

2.1. Frontend-analyse

De frontend is het gezicht van de applicatie. Het zijn de schermen waar gebruikers mee werken, samen met de manier waarop ze werken. Het gekozen framework is Angular versie 21. Angular wordt vergeleken met Blazor en React op basis van de relevante categorieën. Elke categorie krijgt een score van één tot en met vijf, waarna elke optie een totaalscore ontvangt. De volgende categorieën zijn gebruikt om de frameworks te vergelijken: de gebruikte programmeertaal, de beschikbare documentatie en community, de leercurve en hoe relevant het is in België.

De programmeertaal is de hoofdtaal waar rond een framework gebouwd is. De documentatie is alles wat het framework biedt om de start makkelijker te maken, snel specifieke gebruikersaanwijzingen te vinden en hoe uitgebreid en duidelijk deze is. De leercurve duidt aan hoe makkelijk het is om met een framework te starten, en gaandeweg te leren. Een framework dat een 1 krijgt, is zeer moeilijk om zonder ervaring mee te starten. De relevantie duidt aan hoe groot de vraag is naar dit specifieke framework binnen België. De relevantie binnen België kan sterk verschillen van de wereldwijde relevantie.

2.1.1. Scoring

De programmeertaal

Als programmeertaal gebruiken zowel Angular als React TypeScript, een taal die wel 45% van de stemmen behaald heeft tijdens een stemming van StackOverflow (StackOverflow, 2025). In de moderne ontwikkelingswereld is dit een taal die ontzettend veel gebruikt wordt. C#, de taal die Blazor gebruikt, heeft een stemming van 27%, wat bijna de helft is van TypeScript. Ondanks dit gebruikt AllPhi C# & .NET voor de ontwikkeling van hun backends. Eenzelfde taal gebruiken voor alles maakt het een stuk eenvoudiger voor de developer. Alle drie de frameworks ontvangen een vier voor deze categorie. TypeScript is de populairdere taal, maar C# de relevantere, waardoor de score gelijk is.

De documentatie

Tijdens het zoeken naar documentatie werd snel duidelijk dat de beschikbare documentatie voor Blazor een stuk uitgebreider is dan die van React en Angular. Ze bieden alle drie een tutorial aan voor starters, samen met de documentatie, waar die van Blazor simpelweg uitgebreider is. Zowel de documentatie, een tutorial, video's en een volledige cursus zijn beschikbaar en makkelijk te vinden. Hierdoor krijgt Blazor een vier, React en Angular ontvangen een drie.

De leercurve

Omdat Angular en React TypeScript gebruiken, is de leercurve hier hoger dan bij Blazor. Tijdens de studie bij Thomas More is C# veel aan bod gekomen, maar TypeScript alleen tijdens een keuzevak. Hierdoor is Blazor voor deze stage een stuk makkelijker om mee aan de slag te gaan, en voelt het een stuk vertrouwd aan. Ondanks dit heeft elk framework zijn eigen manier van werken, waar de taal zelf een mindergrote invloed op heeft. Alle drie de frameworks ontvangen hier een drie.

De relevantie

Volgens de stemming van StackOverflow is React veruit het meest gebruikte framework, met wel 45% van de developers die hiervoor kiezen (*StackOverflow, 2025*). Angular heeft hier een score van 18%, en Blazor een score van 7%. Ondanks dit vertelde de Regionaal Consultant Coördinator van AllPhi Westerlo, en de stagementor van deze stage, dat Angular bijzonder aantrekkelijk is in België, aantrekkelijker dan React. Hierdoor is de keuze voor Angular door AllPhi een stuk logischer, waardoor het op deze categorie een vijf krijgt. React ontvangt een vier, en Blazor een drie.

2.1.2. Conclusie

Angular behaalde een vier voor de categorie van de programmeertaal, een drie voor documentatie, een drie voor leercurve, en een vijf voor relevantie. Het is een framework waar veel naar gevraagd wordt binnen België, waardoor het zeker relevant is voor een bedrijf zoals AllPhi, en om ervaring mee op te doen tijdens een Belgische stage. Angular heeft een totaalscore behaald van vijftien op twintig.

React behaalde een vier voor de programmeertaal, een drie voor documentatie, een drie voor de leercurve en een vier voor relevantie. Ondanks dat dit het meest gebruikte framework is volgens StackOverflow, is het in België minder aantrekkelijk. React behaalde een totaalscore van veertien op twintig.

Blazor behaalde een vier voor de programmeertaal, een vier voor documentatie, een drie voor leercurve en een drie voor relevantie. Dit is een aantrekkelijk framework voor bedrijven die al gespecialiseerd zijn in .NET & C#. Binnen België moet het framework nog groeien. De totaalscore van Blazor is een veertien op twintig.

Angular past perfect binnen de .NET-omgeving van AllPhi en integreert hier heel goed in. Er zijn veel gelijkenissen tussen .NET en Angular, zoals 'dependency injection' bijvoorbeeld. Zo is het makkelijker om Angular snel te leren kennen en mee te zijn met de gebruikte technologie, en is de app gebouwd met een wereldwijd veelgebruikt framework. Werken met TypeScript is eenvoudig en Angular laat ons snel robuuste frontends bouwen op enterprise-niveau, samen met de ondersteuning van ontzettend veel packages, gebouwd voor JavaScript of TypeScript.

Hieruit kan geconcludeerd worden dat AllPhi een correcte keuze gemaakt heeft voor het frontend framework van deze applicatie. De analyse bevestigt dat dit een relevant framework is om tijdens een Belgische stage aan te leren.

2.2. Backend-analyse

De backend is het brein van de applicatie. Het beheert de data van de applicatie: data manipuleren, nieuwe data opslaan, de juiste data teruggeven... Er zijn verschillende backend frameworks en architecturen om een backend mee te realiseren, maar AllPhi koos voor ASP.NET Core tijdens deze stage.

ASP.NET Core staat bekend als het framework voor grote enterprise-level backends. Het is robuust, kan in veel mogelijke richtingen evolueren en is zeer performant en schaalbaar.

AllPhi is hierin gespecialiseerd, samen met andere Microsoft-omgevingen zoals SharePoint en Azure, waardoor deze keuze bijna niet in vraag gesteld kan worden. Andere backend frameworks zoals Laravel en Node.js zijn een optie, maar vallen buiten de specialisatie van AllPhi. ASP.NET Core is perfect geschikt voor de backend die gebouwd moet worden voor dit project, waardoor er geen vergelijking is opgesteld.

Design patterns

Clean Architecture is een designpatroon dat Microsoft zelf aanraadt voor het bouwen van robuuste en schaalbare backends (*Microsoft Learn, 2023*). Het verdeelt de code in verschillende lagen, die elk zo weinig mogelijk en hele specifieke verbindingen met elkaar hebben, in vooraf vastgestelde richtingen. Dit zorgt

voor code die losgekoppeld is van elkaar, waardoor delen makkelijk aangepast kunnen worden, of zelfs vervangen kunnen worden. Ook lekt de implementatie van een laag moeilijk of zelfs niet in een andere laag.

Mediator is een package dat helpt om de features van de applicatie te onderscheiden van elkaar, met het loskoppelen van code als gevolg (*lucky penny software, 2025*). Het sluit perfect aan op clean architectuur, waardoor deze keuze ontzettend sterk is.

De entiteiten, waar de volledige backend rond gebouwd is, worden gebouwd met Domain Driven Development (DDD). Dit betekent dat elke entiteit zijn eigen regels afdwingt, weet in welke staat hij mag bestaan en wanneer niet, en zorgt ervoor dat dit daadwerkelijk zo is. Hierdoor is de validatie van een entiteit gecentraliseerd in de entiteit zelf, en lekken de domainregels niet in andere lagen.

Door het bekijken van deze principes, patronen en framework, is er snel tot het besluit gekomen om dit niet verder in vraag te stellen.

2.3. Projectmanagementtool

Jira is de tool die Thomas More gebruikt voor projectmanagement. Door open te staan om een nieuwe tool te ontdekken, is er plaats om nieuwe ervaringen op te doen. Veelgebruikte projectmanagementtools zijn bijvoorbeeld GitHub en DevOps.

De start van het project was met GitHub Boards omdat GitHub een makkelijke en gratis manier biedt om code in de cloud op te slaan via repositories, en veel gebruikt wordt door scholieren, individuele developers en zelfs grotere bedrijven. (*Github, z.d.*) Ervaring met deze tool is een zeer gewaardeerde skill, ook binnen AllPhi, waar GitHub Enterprise gebruikt wordt.

Er was tijdens de start van de stage nog geen DevOps-repository beschikbaar, waardoor GitHub de meest logische keuze leek, die ook gebruikt is tijdens de start van het project. Omdat DevOps een volwassenere tool is en frequenter gebruikt wordt binnen AllPhi, is er op het einde van week twee van de stage een DevOps-repository ter beschikking gesteld.

DevOps is dus de projectmanagementtool die gebruikt zal worden doorheen dit project, zowel voor versiecontrole als voor sprintplanningen en backlog.

2.4. Besluit

De applicatie zal bestaan uit een frontend die gerealiseerd wordt met Angular. Een zeer relevante tool binnen België. Het onder de knie krijgen van Angular zal gaandeweg tijdens het ontwikkelen gebeuren, samen met het regelmatig herwerken van code om de gehele applicatie aan de hoogste standaard te houden.

De backend wordt gebouwd met ASP.NET Core. Dit zorgt voor een enorme hoeveelheid kennis en ervaring van consultants die steun en hulp kunnen bieden tijdens de ontwikkeling van het stageproject. Ook is dit een framework dat veel gebruikt wordt wereldwijd, wat het een zeer relevante skill maakt.

Projectmanagement is belangrijk om een project tot een goed einde te brengen, en een correcte tool speelt een even belangrijke rol. DevOps blijft binnen Azure, wat een specialisatie is van AllPhi. Dit is een relevante skill, zeker binnen de ontwikkelingswereld van Microsoft, waar .NET deel van uitmaakt.

3. Backend development

De backend is, zoals voorheen besproken, gestructureerd volgens het Clean Architecture-principe, waarbij de code opgesplitst is in verschillende lagen om zo losse koppeling te creëren in de code. Er zijn vier grote lagen binnen de backend, namelijk de API-laag, de applicatie-laag, de infrastructuur-laag en de domain-laag. Deze lagen hebben elk hun focus: bij de domain-laag zijn dit de entiteiten, bij Infrastructuur is dit alles rond de database en blob storage, bij de applicatie zijn dit de verschillende business requirements, en de API is het aanspreekpunt van de backend, en zorgt verder ook voor security en andere globalere zaken zoals logging.

Door clean architecture toe te passen, is de backend verdeeld in verschillende projecten. Dit houdt elk project simpel, met de focus op de toegevoegde functionaliteit. Er is gekozen voor verschillende projecten per laag in plaats van verschillende folders per laag, zodat clean architecture wordt afgedwongen, en dit geen keuze meer is. (*Figuur 1*)

Dit biedt de zekerheid dat de juiste projecten elkaar kennen, maar geen weet hebben van de projecten waar ze niets mee te maken hebben. Zo heeft de application-laag weet van de domain-laag, maar niet omgekeerd. Door folders te kiezen wordt het afdwingen hiervan moeilijk, omdat files in hetzelfde project altijd weet hebben van elkaar, en dus per ongeluk gebruikt kunnen worden op plaatsen waar dit niet gewild is.

Er zijn ook testprojecten die alle logica van de domainentiteiten testen via unittests, en de gehele applicatie via integratietests. (*Figuur 2*) xUnit is gebruikt als framework om de testen te schrijven binnen dit project. (*xUnit.net*, z.d.)

Dit hoofdstuk zal de gehele flow van de backend uitleggen en refereren naar de verschillende lagen waar relevant. De verschillende tools die gebruikt zijn, worden ook aangehaald.

3.1. Domain-laag

Zoals eerder gezegd leven hier al de entiteiten en hun logica. Elke entiteit zorgt voor zijn eigen validatie, waardoor ze alleen bestaan met geldige data. Dit patroon staat bekend als DDD, of Domain Driven Development. Een simpel voorbeeld hiervan is de gebruiker-entiteit. Elke gebruiker heeft data, zoals een gebruikersnaam, een wachtwoord, een e-mailadres... De logica in die entiteit zorgt ervoor dat een gebruiker alleen aangemaakt kan worden als de gebruikersnaam niet leeg is, als het wachtwoord ingevuld is en minimaal 8 karakters lang is...

Elke entiteit is gebouwd als een 'sealed class'. Zo kan er worden gegarandeerd dat een gebruiker gesloten is, en niet openstaat voor overerving. Ook zijn alle properties van elke entiteit gesloten voor manipulatie van buiten de entiteit. Op deze manier kan de manipulatie van de entiteit intern beheerd worden, en kunnen er alleen juiste instanties gemaakt worden. (*Figuur 3-4*)

De e-mail is een 'value object'. Een e-mail kan dus aangemaakt worden als een 'Email' object, niet als een simpele string. Hierdoor bevat een e-mail zelf zijn eigen logica en zorgt er zelf voor dat elke instantie van dit object correct is. Deze keuze is gemaakt om de logica uit de gebruiker-entiteit te halen, en te splitsen in een apart object zodat de gebruiker alleen zijn eigen logica bevat, en niet te specifiek gaat. (*Figuur 5*)

Verder zijn de entiteiten opgebouwd met het 'Result Pattern'. In plaats van een exceptie te gooien als een record van een entiteit aangemaakt wordt met foutieve data, geeft de entiteit een resultaat terug. Dit resultaat kan succesvol zijn, en een nieuw record van die entiteit teruggeven. Maar dit resultaat kan ook niet succesvol zijn en validatiefouten bevatten. Deze validatiefouten zijn geen exceptie, dus kunnen mooi

meegeleverd worden wanneer de API de data terugstuurt. De frontend weet zo ook vooraf wat te verwachten als er validatiefouten zijn, en zo worden veel onnodige checks vermeden. (Figuur 6)

3.2. Applicatie-laag met MediatR

MediatR is een package die gebruikt wordt om de API-laag los te koppelen van de applicatie-laag. Elke feature bestaat uit 3 verschillende delen, namelijk: het 'request', dit bevat de data waarmee gewerkt moet worden, bevat eventuele parameters... De 'validator' valideert het request waarmee gewerkt wordt. De 'handler' gebruikt het gevalideerde request om een business case uit te voeren, zoals 'Create User'.

Wanneer een endpoint aangesproken wordt, zal er met de meegeleverde data een request aangemaakt worden, waarna dit verzonden wordt met MediatR. Een MediatR-pipeline zorgt ervoor dat dit request onderschept wordt, en eerst door de validator gaat. Zo kunnen validatiefouten snel opgepikt worden, waarna de gebruiker op de hoogte gebracht kan worden. De fouten worden vroegtijdig onderschept, en de data is niet gemanipuleerd. (Figuur 7)

Als er geen fouten zijn, gaat de handler met het request de verwachte business case uitvoeren, zoals het ophalen van een gebruiker voor een Id, een gebruiker updaten met de data uit het request... Wanneer de handler klaar is wordt de data teruggestuurd naar de API, en bereikt zo terug de eindgebruiker.

De API-laag en de applicatie-laag zijn volledig losgekoppeld van elkaar. Elke feature heeft zijn request, wat nauwelijks verandert, omdat dit specifiek is voor die business case. De logica die in de handler zit, of de validatie regels, kunnen regelmatig veranderen of zelfs volledig herwerkt worden. Door het gebruik van MediatR moet de API nu niet mee aanpassen. Dit is losse koppeling.

In deze applicatie is MediatR toegepast voor elk CRUD-endpoint. Een concreet voorbeeld is het 'Create Training' command: het request bevat de training-data, de validator controleert of de datum niet in het verleden ligt, en de handler voert de businesslogica uit. Een trade-off van MediatR, of CQRS (Command Query Responsibility Segregation) in het algemeen, is dat dit bij GET-endpoints soms als overkill aanvoelt, maar om de codebase uniform te houden is dit consistent gebleven voor alle endpoints. (Figuur 8)

3.3. Opslaan van data

De infrastructuur-laag zorgt voor het juist opslaan van files en data. De data bestaan uit de gebruikers, de trainingen of certificaten, de teams... en worden opgeslagen in een SQL-database. Files, zoals een PDF of PNG, kunnen geupload worden samen met een training of certificaat, en worden opgeslagen in de Azure Blob Storage. (Microsoft Learn, 2024)

Entity Framework Core

Entity Framework Core helpt ons bij het manipuleren van data, door het mappen van domainentiteiten naar database-entiteiten eenvoudig te maken. Via LINQ kunnen query's makkelijk geschreven worden om bijvoorbeeld de data in een lijst te filteren of de juiste data uit de database te halen zonder zelf SQL te hoeven schrijven. (Figuur 9)

Entity Framework biedt mogelijkheden om de manipulatie van data in de database te onderscheppen, om zo acties te manipuleren. Een voorbeeld hiervan is het 'soft deleten' van data. In plaats van het record effectief uit de database te verwijderen, onderschept het elke actie. Hierna wordt er gekeken of de entiteit verwijderd mag worden of niet, en wordt de 'IsDeleted'-property van de data aangepast in plaats van het record effectief te verwijderen. Hierdoor kan de code gewoon geschreven worden zoals normaal om data te verwijderen, maar wordt de echte implementatie van de delete in de interceptor bepaald. (Figuur 10)

Repositories

Repositories bevatten de entiteit-specifieke logica voor het manipuleren van hun data. De repositories worden aangesproken in de applicatie-laag, door de handler van een command. Een command duidt aan dat data gemanipuleerd moet worden in deze use case, dus niet specifiek het afhalen van data.

Een 'Certificaat Update' command duidt aan dat een certificaat geüpdatet moet worden. Het command bevat het id van het certificaat en de nieuwe data. De handler zal met dit command de repository van certificaten aanspreken, om zo dit certificaat te updaten. Zoals eerder gezegd staat een command voor datamanipulatie, maar natuurlijk moet er ook data afgehaald worden voor er data aangepast kan worden. De repository is de plaats waar dit gebeurt. Het haalt het juiste certificaat af, valideert of de nieuwe data geldig is, en update het record als dit gepast is. (*Figuur 11*)

Om losse koppeling van code te creëren tussen de infrastructuur- en applicatie-laag, heeft elke repository zijn eigen interface. De applicatie-laag weet welke acties beschikbaar zijn door de interfaces, maar is niet geïnteresseerd in hoe deze geïmplementeerd zijn. De implementatie bevindt zich in de repositories zelf. Hierdoor kan een volledige repository-implementatie veranderd worden, zonder dat de applicatie-laag hier weet van heeft. (*Figuur 12*)

3.4. Security

Tijdens de eerdere fases van de stage is er een eigen authenticatie ontwikkeld met gebruik van JSON Web Tokens (JWT). Wanneer een gebruiker succesvol inlogt, zal er een JWT aangemaakt worden voor deze gebruiker, die een dag geldig zal blijven. Dit JWT is nodig voor het gebruiken van de applicatie zelf, en voor het gebruiken van de backend. Bij elk request dat binnenkomt in de API wordt dit token verwacht en grondig gevalideerd, zodat alleen de correcte gebruikers bij de correcte data kunnen. Hiervoor wordt de ASP.NET Core Autorisatie-package gebruikt, die ons attributen ter beschikking stelt om deze checks eenvoudiger af te handelen.

De backend maakt bij login een token aan voor de gebruiker die wil authenticeren. Zo kan het token niet aangepast worden in de frontend, en moeten er geen geheime sleutels blootgesteld worden. Deze genereert de tokens met vooraf ingestelde data, samen met de gebruiker specifieke data, zoals het id en de rol.

Tegen het einde van de stage zal het inloggen via gebruikersnaam en wachtwoord, en het genereren van een eigen JWT, vervangen worden door authenticatie via Azure Entra ID (*Microsoft Learn, 2024*) AllPhi gebruikt deze manier van authenticatie voor al hun interne projecten. Zo blijft het intern inloggen makkelijk en vertrouwelijk.

Echter heeft dit gevolgen voor de huidige staat van de applicatie. De gebruiker-entiteit moet herwerkt worden: zowel de gebruikersnaam als het wachtwoord zijn nu overbodig, en dus verwijderd. Er is een extra id toegevoegd, die uit Azure AD komt, die we gebruiken om de ingelogde Azure-gebruiker te linken aan een gebruiker van de applicatie.

3.5. Orkestratie

De API-laag is het project dat de backend start. De database en de blob storage moeten gestart worden en draaien in een dockercontainer. De frontend zelf is ook een project dat gestart moet worden voordat deze gebruikt kan worden. Om dat allemaal makkelijk en tegelijk te doen, wordt Aspire gebruikt. Door Aspire gaan alle projecten in een correcte volgorde opstarten, zodat projecten wachten met opstarten tot hun connecties eerst gestart zijn. De backend wacht bijvoorbeeld tot de database en de blob storage gestart zijn, voordat deze zelf opstart. Aspire zorgt zelf voor het correct zetten van de environmentvariabelen, waardoor vermeden kan worden dat de applicatie werkt bij sommige developers, maar bij andere niet. (*Microsoft Learn, 2024*)

Aspire werkt vanuit een .NET-project, de AppHost. Hierdoor worden de verschillende projecten geconfigureerd via C#-code, wat het uitzettend aantrekkelijk en makkelijk maakt voor een developer. Per project wordt hier geconfigureerd wat het nodig heeft om met succes op te starten. Ook kunnen specifieke poorten geconfigureerd worden voor de projecten en containers. (*Figuur 13*)

Je kan connecteren met een database via zijn poort, maar als deze bij elke startup aanpast, moet dit bij elke applicatieaanpassing opnieuw gedaan worden. Aspire biedt hier de oplossing om de container te laten draaien op een vaste poort, zodat er maar één keer geconnecteerd moet worden. Ook de frontend wordt elke keer opengesteld via dezelfde poort. Zo blijven de opgeslagen variabelen in de lokale opslag van de browser staan.

3.6. Uitdagingen

De architectuur van de backend maakt alles heel duidelijk, performant en schaalbaar, maar is een grote taak om op te zetten. Het vraagt goed nadenken over de keuzes, zoals de opbouw van entiteiten en gebruikte patronen. Slechte keuzes kunnen grote invloed hebben op de uiteindelijke staat van de backend, en hoe bruikbaar en schaalbaar deze daadwerkelijk zal zijn.

Clean Architecture bracht soms moeilijkheden met zich mee, zoals op een correcte manier de logica verdelen tussen de verschillende lagen. Het bestuderen van andere projecten samen met het opzoeken naar relevante artikels over Clean Architecture maakte het iets eenvoudiger om keuzes te maken. Ook de aanwezigheid van consultants in het kantoor van AllPhi speelde een rol, omdat zij altijd beschikbaar zijn om te sparren over een onderwerp, of om een perspectief van een professional te krijgen over een keuze die gemaakt moet worden.

Het opzetten van integratietesten was een grote taak, omdat dit volledig nieuw is. Het draaien van de backend in een testomgeving, samen met de database en blob storage maakte het een moeilijke taak. Er is veel opzoekwerk gedaan naar integratietesten, en een referentieproject van AllPhi hielp sterk om de juiste beslissing en designkeuzes te maken.

4. Frontend development

Zoals eerder besproken, is er gekozen om Angular te gebruiken als frontend framework vanwege de hoge relevantie binnen de Belgische markt en de relevantie binnen AllPhi. Dit bracht echter een steile leercurve mee, omdat Angular en TypeScript volledig nieuwe technologieën waren. Het design van een Angular-project was hierdoor ook nieuw en bracht redelijk wat opzoekwerk met zich mee.

Gebruikers hebben een rol die bepaalt hoe zij gebruik kunnen maken van de applicatie. De verschillende opties zijn een employee, een manager of een administrator. Bijvoorbeeld een employee is een consultant bij AllPhi, die de applicatie zal gebruiken om een record op te slaan van een training of een certificate. Hij zal dus alleen de pagina's kunnen bereiken en gebruiken die van belang zijn voor een consultant. Een manager heeft diezelfde twee pagina's, plus de managerspecifieke pagina's.

4.1. Architectuur

Bij .NET-frontend frameworks, zoals Blazor of Maui, wordt er gewerkt met een MVVM-patroon, terwijl in Angular een component-based of feature-based architectuur de norm is. Tijdens het ontwikkelen van de frontend zijn verschillende delen meermaals herschreven, om beter te voldoen aan de gekozen architectuur, om dubbele code tot een minimum te houden, en om code zo herbruikbaar mogelijk te maken. (*Microsoft Learn, 2024; Angular Team, 2025*)

4.1.1. De voordelen van componenten

Voor deze stage is de keuze gemaakt om via een component-based architectuur de frontend te ontwikkelen. Dit betekent dat pagina's bestaan uit verschillende componenten die eventueel herbruikbaar zijn voor andere pagina's. Dit zorgt ervoor dat elke component zijn eigen logica en opbouw heeft, wat zowel de pagina's zelf als de individuele componenten compacter en overzichtelijker houdt.

De dropdowncomponent is een perfect voorbeeld van een herbruikbare component met specifieke logica. De dropdown ziet eruit als een inputveldje, maar wanneer men erop klikt, verschijnt er een pop-up met de verschillende selecteerbare opties. De component zelf bevat alle logica voor het openen van de pop-up, het selecteren van een optie, en de nodige logica die nodig is om de hoeveelheid selecteerbare opties te beperken, bijvoorbeeld. Dit maakt het gebruiken van deze component eenvoudig: op de pagina's waar de component gebruikt wordt, worden de beschikbare opties meegeleverd, en wordt de huidige geselecteerde optie uitgelezen.

Door deze component herbruikbaar te maken wordt er voorkomen dat dropdown-specifieke logica gekopieerd moet worden van pagina naar pagina. Ook als er een bug is, kan dit aangepast worden op een centrale plek. (*Figuur 14 – 15*)

4.1.2. Services & facade-pattern

Verschiedende services worden gebruikt om de API aan te spreken, met een service per entiteit. Hierdoor legt elke service de focus op maar één specifieke entiteit, waardoor de services overzichtelijker zijn en alleen gebruikt kunnen worden wanneer deze entiteit en zijn functionaliteiten nodig zijn. Het 'Facade Pattern' is gebruikt om een centrale plek te voorzien van alle nodige functies en data van een entiteit. Elke facade spreekt de service aan van zijn entiteit, en biedt functies aan die de data gebruiken of een specifiekere use case die de API zelf niet aanbiedt. Een voorbeeld hiervan is het on-the-spot filteren van al via de API afgehaalde datalijsten.

Certificaten ophalen of manipuleren gebeurt via de certificaat-service. Een service bestaat uitsluitend voor het ophalen van deze data en het versturen van commands voor de manipulatie van certificaten, te centraliseren. De certificaat façade biedt een laag aan boven op deze service. Hij gaat, zoals hierboven besproken, om met de errors die in service voor kunnen komen. (*Figuur 16*)

Bijvoorbeeld: een certificaat voor een bepaald id is niet gevonden, waardoor de backend een http-code 404 'not found' retourneert. Een webapplicatie, zoals deze met Angular, ziet dit als een error. De backend weet wat dit betekent: de gebruiker heeft iets verkeerd ingegeven, maar zowel de app als de gebruiker weten dit niet. De facade zal deze error opvangen en een toast tonen aan de gebruiker die duidelijk maakt wat er fout gegaan is.

Voor certificaten biedt de facade ook een optie aan om een certificaat te vinden op basis van een id bijvoorbeeld. Dit is functionaliteit die meermaals gebruikt wordt doorheen de applicatie, dus de logische stap is genomen om dit in de facade te zetten, waar de data in eerste instantie ook uitkomt. Dit vermindert dubbele code, en zorgt voor een centrale plek om deze logica aan te passen. (*Figuur 17*)

Door het gebruik van facades zit er ook geen servicelogica in de pagina's. De facades bepalen hoe data gemanipuleerd wordt, en wachten op de API. De pagina's kiezen wanneer deze data nodig is, sturen het signaal naar de facades om een actie te doen, maar gaan zelf verder met hun logica. Via de facade kunnen ze de data tonen, die ingeladen wordt wanneer die klaar is. Ook dit zorgt ervoor dat deze logica niet op meerdere pagina's dubbel staat, maar gecentraliseerd is.

4.1.3. Entiteiten als frontend models

De verschillende entiteiten van de backend leven hier als models. Elke entiteit kan verschillende modellen hebben, die gebruikt worden om de data op een specifieke manier te manipuleren. Zo heeft de gebruiker-entiteit bijvoorbeeld een model voor het afhalen, een model voor het aanmaken en een model voor het bewerken van een gebruiker.

Ook de responses die verwacht worden van de API zijn simpele generieke models, die gebruikt worden in de services. Samen met een model voor de errors wordt er omgaan met de verschillende datamogelijkheden die de backend kan terugsturen. Errors worden correct afgehandeld, en verwachte data kan gebruikt worden.

Ook zijn er frontendspecifieke modellen, die verschillende data bijhouden, wat in de backend apart kan zitten. Een voorbeeld is een model voor de files. Files worden opgehaald via een SAS-token, zoals voorheen besproken. Dit bevat de naam van de file en de link naar de file, in één. In de backend wordt deze URL teruggestuurd, maar in de frontend wordt dit opgeslagen in een model met een naam, een filetype (pdf, jpg...) en de URL naar de file. Hierdoor kan er makkelijker omgegaan worden met de files in de frontend, door direct de naam te kunnen gebruiken om te tonen, en de UI te manipuleren op basis van het filetype. (*Figuur 18*)

Er is gekozen om interfaces te gebruiken voor het bouwen van deze models, zoals aangeraden door TypeScript en Angular zelf. (*Typescriptlang, Z.D.*)

4.2. Belangrijke pagina's

De applicatie bestaat uit verschillende pagina's, maar er zijn drie grote soorten pagina's die invloed hebben op hoe de applicatie werkt. Alle drie deze pagina's worden hieronder in detail besproken, waarom deze gebouwd zijn op de manier dat ze gebouwd zijn, en welke invloed ze hebben op de applicatie.

4.2.1. Data Tables

De accountmanagementpagina is een pagina met een grote tabel waar elke rij een account representeert, en elke kolom de data van de gebruiker-entiteit. Zoals eerder gezegd kan dit bijvoorbeeld de gebruikersnaam, het wachtwoord, het e-mailadres zijn. Via deze tabel kunnen de verschillende accounts beheerd worden, kunnen er nieuwe accounts toegevoegd worden of kunnen accounts verwijderd worden. De tabel biedt filteropties om zo makkelijk te tonen wat er verwacht wordt, of om specifiek een account te zoeken bijvoorbeeld.

Het grootste deel van de pagina's van de Training & Certificate Tracker zijn datatabellen. Deze tabellen bieden veel mogelijkheden en functionaliteiten op een centrale plek, wat het makkelijk maakt om bijvoorbeeld trainingen en certificaten te beheren, wat het doel is van deze applicatie.

Hierdoor is dit de perfecte component om volledig herbruikbaar te ontwerpen. Hij bevat zelf weinig tot geen logica, maar is puur bedoeld voor het eenvoudig en snel opzetten van een tabel. Dit zorgt er ook voor dat alle tabelpagina's hetzelfde functioneren, waardoor deze consistentie doorheen de applicatie behouden wordt. (*Figuur 19-21*)

4.2.2. Team skill matrix

De team skills matrix is een component die een tweedimensionale matrix gaat genereren op basis van certificaattypes en de werknemers in het team van een manager. Zo heeft de manager een at-a-glance-overzicht van de certificaatstatussen van zijn consultants. Hij kan zo doelgericht zoeken naar consultants met relevante certificaten die bijvoorbeeld op de bank zitten. Als hij iets interessants opmerkt, biedt de matrix aan te focussen op één specifiek certificaatype, of de mogelijkheid om direct te navigeren naar de detailpagina van de consultant zelf. Op deze detailpagina kan de manager de certificaten in detail bekijken, alsook de trainingen en de persoonlijke info van de consultant. (*Figuur 22-23*)

4.2.3. Dashboard

De dashboardpagina is de homepage voor employees en administrators. Ze geven de gebruiker een top-down overview van de relevante status binnen AllPhi.

Een employee ziet via vier tegels hoeveel trainingen hij gevolgd heeft, hoeveel van zijn certificaten actief zijn, hoeveel er binnenkort gaan vervallen en hoeveel er al vervallen zijn. Via een staafdiagram kan de employee bekijken hoeveel trainingen hij per maand gehaald heeft, van de afgelopen 12 maanden. Deze data zijn zeer handig om te weten of het tijd is om bij te scholen.

Via een donutdiagram krijgt de employee een snel overzicht dat toont hoeveel procent van zijn certificaten actief, aflopend of vervallen zijn. Deze data worden in nummers getoond in de tegels, maar deze grafiek geeft dit visueel nogmaals mee, waardoor het makkelijker is om die nummers te begrijpen.

De managers zien dezelfde tegels en diagrammen, maar met de totaalnummers van alle consultants in hun team. De administrators zien dit van alle consultants binnen AllPhi. (*Figuur 24*)

5. Persoonlijke groei

Tijdens deze individuele stage is er veel bijgeleerd over de technologieën die gebruikt zijn, maar ook de soft skills zijn verbeterd. Door individueel te werken, werd er geleerd hoe een analyse om te zetten in een werkende applicatie. Het maken van belangrijke keuzes kan snel besproken worden in een team, maar als individu is het zeer belangrijk om de antwoorden hierop zelf te vinden, te noteren, eventueel te testen in een proof-of-concept, en zo met de vraag en antwoorden naar een collega gaan. Iedereen werkt aan zijn eigen project, en via deze manier van vragen is de collega snel mee met het probleem, en door de aangeboden antwoorden is waardevolle input geven veel makkelijker.

Tijdens deze stage hebben er drie code reviews plaatsgevonden, en een finale demo voor de Consultant Coördinator van Westerlo, en de General Manager van AllPhi. Tijdens de code review is er geleerd hoe de nieuwe features kort maar bondig voor te stellen, en het verduidelijken van de bijhorende code. Ook het krijgen van feedback is belangrijk, van zowel de presentatie als de implementatie. Dit individueel noteren en ermee aan de slag gaan is uitdagender dan in team, maar een zeer mooie skill om goed onder de knie te krijgen.

Tijdens de finale demo voor niet-technische collega's is het belangrijk om tegen een goed tempo de belangrijkste zaken aan het licht te brengen. Dit wil niet zeggen dat het technische genegeerd moet worden, maar dat dit op de juiste manier aan bod moet komen.

Al bij al was dit een zeer leervolle stage, zowel op technisch als op persoonlijk vlak. Er is gespeeld met nieuwe technologieën, code is herschreven om up-to-date en performant te blijven, en combinaties van frameworks zijn ontdekt voor het optimaal bouwen van een webapplicatie. Ook heeft de stage ervoor gezorgd dat de nodige skills aanwezig zijn om een analyse als full-stack developer te realiseren, met het nauw samenwerken met de klant, het correct analyseren en het in vraag stellen van gemaakte keuzes.

6. Besluit

Tijdens deze stage bij AllPhi is de 'Training & Certificate Tracker' gerealiseerd. Dit is een applicatie die het beheren van consultanttrainingen en certificaten centraliseert en digitaliseert. Het doel van dit project was om een applicatie te bouwen die zowel de consultants als overhead kunnen gebruiken, om een centrale plek te hebben om trainingen en certificaten te beheren, en inzicht te krijgen in de status van AllPhi. Dit doel is bereikt door een moderne, robuuste en schaalbare applicatie te bouwen, die gebruiksvriendelijk is en voldoet aan de eisen van AllPhi.

Clean architecture, ASP.NET Core en Angular is de basis waarop deze applicatie gebouwd is. Samen met het gebruiken van DDD en MediatR garandeert de backend een codebase die schaalbaar en overzichtelijk is. De frontend, gebouwd met Angular 21, is een uitstekende keuze voor een IT-toepassing, zeker in België, en past perfect in de .NET-toepassingen van AllPhi. Door de component-based architectuur biedt de frontend een consistente ervaring voor de gebruikers, en een onherleidbare codebase voor developers.

De stap die de applicatie daadwerkelijk bruikbaar maakte voor AllPhi, was de integratie van Azure AD en het vervangen van de custom JWT-authenticatie. De eigen implementatie was functioneel, maar door Azure AD te gebruiken wordt de integratie met de AllPhi-infrastructuur compleet en is het een applicatie op enterprise-niveau. Het gebruik van Aspire voor de orkestratie van de applicatie was essentieel om de ontwikkeling vlot te doen verlopen, en zorgt ervoor dat andere developers makkelijk kunnen starten met deze applicatie.

Ook op persoonlijk vlak was deze stage een succesvolle leerervaring. Het individueel werken zonder een mentor die te alle tijden technisch kan bijsturen, zorgde voor het leren om analyses zelfstandig door te nemen en kritisch te bekijken, alsook technische keuzes zelfstandig te verantwoorden. Individueel de analyse omzetten in een werkende applicatie zorgde voor een verbeterde full-stack kennis, en het verbeteren van probleemoplossend en architecturaal denken.

Door de code review en meetings met zowel technische als niet-technische collega's van AllPhi was het nodig om het presenteren van technische details op de juiste manier voor te brengen. Op een toegankelijke manier complexe of technische concepten overbrengen is een skill die hier zeker aan bod gekomen is.

De basis is ontwikkeld en de applicatie voldoet aan alle noden die AllPhi momenteel heeft. De gekozen architecturen maken het verder uitbreiden van de applicatie zeer eenvoudig. Deze stage was een succesvolle technische en persoonlijke uitdaging, die tot een mooie en functionele applicatie geleid heeft. De 'Training & Certificate Tracker' is niet enkel een nieuwe tool voor AllPhi, maar ook een bewijs van de vaardigheden die nodig waren voor de applicatieontwikkeling ervan.

LITERATUURLIJST

StackOverflow (2025), Most popular technologies.

<https://survey.stackoverflow.co/2025/technology>

Typescriptlang (z.d.), Types vs Interfaces.

<https://www.typescriptlang.org/play/?#example/types-vs-interfaces>

Microsoft Learn (2023), Common web application architectures.

<https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>

Mediatr (Lucky Penny Software, 2025), The simple yet powerful mediator.

<https://mediatr.io/#features>

xUnit.net (z.d.). xUnit.net: A free, open source, community-focused unit testing tool.

<https://xunit.net/?tabs=cs>

Microsoft Learn (2024). .NET Aspire.

<https://aspire.dev/>

Angular Team (2025). Angular Documentation.

<https://angular.dev/>

GitHub. (z.d.). GitHub Boards.

<https://github.com/features/issues>

Microsoft Learn (2024). Azure Blob Storage.

<https://learn.microsoft.com/en-us/azure/storage/blobs/>

Microsoft Learn (2024). Azure Entra ID.

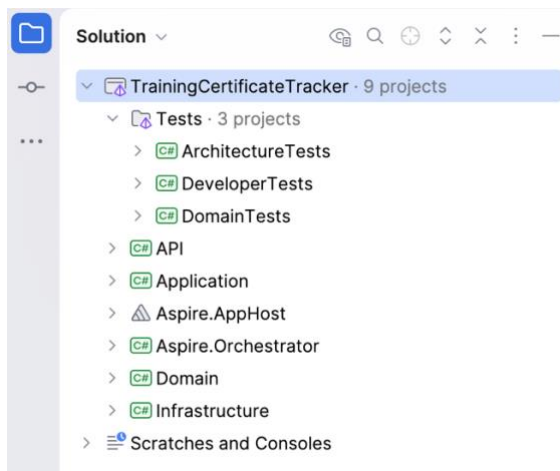
<https://learn.microsoft.com/en-us/entra/identity/>

Microsoft Learn (2024). Model-View-ViewModel (MVVM).

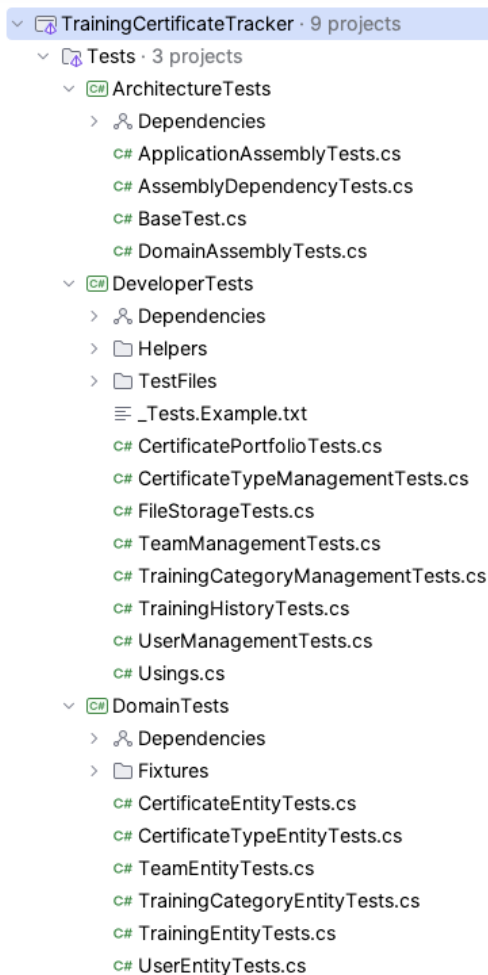
<https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>

BIJLAGEN

Figuur 1: Clean Architecture lay-out van de backend



Figuur 2: Test folders voor architectuur, domain en integratie testen



Figuur 3: Sealed User class met zijn eigenschappen

```
public sealed class User : Entity, ISoftDeletable
{
    [MauroL.AllPhi]
    private User()
    {
    }

    [21 usage] [MauroL.AllPhi]
    private User(
        string providerUserId,
        string fullName,
        Email email,
        UserRole role,
        Guid? teamId
    )
    {
        ProviderUserId = providerUserId;
        FullName = fullName;
        Email = email;
        Role = role;
        TeamId = teamId;
    }

    [21 usage]
    public string ProviderUserId { get; private init; } = null!;
    [39 usage]
    public string FullName { get; private set; } = null!;
    [15 usage]
    public Email Email { get; private set; } = null!;
    [19 usage]
    public UserRole Role { get; private set; } = UserRole.Employee;
    [17 usage]
    public Guid? TeamId { get; private set; }

    [13 usage]
    public Team? Team { get; private set; }

    [1 usage]
    public IReadOnlyCollection<Team> ManagedTeams { get; private set; } = new List<Team>();
    [7 usage]
    public IReadOnlyCollection<Training> Trainings { get; private set; } = new List<Training>();
    [15 usage]
    public IReadOnlyCollection<Certificate> Certificates { get; private set; } = new List<Certificate>();
}
```

Figuur 4: Create methode voor gebruiker die een correcte instantie teruggeeft

```
public static Result<User> Create(
    string providerUserId,
    string fullName,
    string email,
    UserRole role,
    Guid? teamId
)
{
    var errors :List<Error> = ValidateInput(
        fullName,
        providerUserId,
        role
    );

    var emailResult = Email.Create(email);
    if (!emailResult.IsSuccess)
        errors.AddRange(emailResult.Errors);

    if (errors.Count != 0)
        return Result<User>.Fail(errors);

    var user = new User(
        providerUserId,
        fullName,
        emailResult.Value,
        role,
        teamId
    );

    return Result<User>.Ok(user);
}
```

Figuur 5: Email Value Object met eigen validatie

```
public sealed class Email : ValueObject
{
    private const string EmailRegex = @"^[^\s]+@[^\s]+\.[A-Za-z]+$";
    // 16 usages
    public string Value { get; }

    // 1 usage  ↳ MauroL.AllPhi
    private Email(string value)
    {
        Value = value.Trim().ToLower();
    }

    // 2 usages  ↳ MauroL.AllPhi +1
    public static Result<Email> Create(string email)
    {
        if (string.IsNullOrWhiteSpace(email))
            return Result<Email>.Fail([Error.InvariantViolation("Email address cannot be empty!")]);

        if (!Regex.IsMatch(input: email, pattern: EmailRegex))
            return Result<Email>.Fail([Error.InvariantViolation("Not a valid email address!")]);

        var newEmail = new Email(email);
        return Result<Email>.Ok(newEmail);
    }

    // 1 usage  ↳ MauroL.AllPhi
    public override string ToString() => Value;

    // 0+1 usages  ↳ MauroL.AllPhi
    public override int GetHashCode()
    {
        return GetEqualityComponents() // IEnumerable<object?>
            .Select(x: object? => x != null ? x.GetHashCode() : 0) // IEnumerable<int>
            .Aggregate((x: int, y: int) => x ^ y); // int
    }

    // 1+1 usages  ↳ MauroL.O4
    public override bool Equals(object? obj)
    {
        return obj is Email other && Equals(other);
    }

    // 1 usage  ↳ MauroL.AllPhi
    protected override IEnumerable<object?> GetEqualityComponents()
    {
        yield return Value;
    }
}
```

Figuur 6: Result class voor succes en mislukt

```
public class Result
{
    // 103 usages
    public bool IsSuccess { get; protected init; }
    // 93 usages
    public IReadOnlyList<Error> Errors { get; protected init; } = [];

    // 26 usages  ↳ MauroL.AllPhi
    public static Result Ok()
        => new() { IsSuccess = true, Errors = [] };

    // 33 usages  ↳ MauroL.AllPhi
    public static Result Fail(IEnumerable<Error> errors)
        => new() { IsSuccess = false, Errors = errors.ToList() };
}

// 72 usages  ↳ MauroL.AllPhi  ↳ 14 exposing APIs
public class Result<T> : Result
{
    // 59 usages
    public T Value { get; private init; } = default!;

    // 17 usages  ↳ MauroL.AllPhi
    public static Result<T> Ok(T value)
        => new() { IsSuccess = true, Value = value, Errors = [] };

    // 27 usages  ↳ MauroL.AllPhi
    public new static Result<T> Fail(IEnumerable<Error> errors)
        => new() { IsSuccess = false, Value = default!, Errors = errors.ToList() };
}
```

Figuur 7: Create training endpoint die mediatr aanspreekt

```
[HttpPost]
[Authorize(Roles = $"{nameof(UserRole.Employee)};{nameof(UserRole.Manager)}")]
[ProducesResponseType<SuccessResponse<TrainingResponseDto>>](statusCode: StatusCodes.Status201Created)]
[ProducesResponseType<FailureResponse>](statusCode: StatusCodes.Status400BadRequest)]
[ProducesResponseType<FailureResponse>](statusCode: StatusCodes.Status401Unauthorized)]
[ProducesResponseType<FailureResponse>](statusCode: StatusCodes.Status403Forbidden)]
[ProducesResponseType<FailureResponse>](statusCode: StatusCodes.Status404NotFound)]
[ProducesResponseType<FailureResponse>](statusCode: StatusCodes.Status500InternalServerError)]
[1 usage  MauroL AllPhi «1
public async Task<ActionResult<IApiResponse>> CreateAsync(
    [FromBody] TrainingCreateDto trainingDto,
    CancellationToken cancellationToken
)
{
    var canCreateTraining :AuthorizationResult = await _authorizationService // IAuthorizationService
        .AuthorizeAsync(User, trainingDto.UserId, nameof(CanCreateTrainingRequirement)); // Task<AuthorizationResult>
    if (!canCreateTraining.Succeeded)
        return Forbid();

    var command = new CreateTrainingCommand(trainingDto);
    var createResponse = await _mediator.Send(command, cancellationToken);

    return createResponse.Success
        ? Created(uri: nameof(CreateAsync), SuccessResponse<TrainingResponseDto>.Create(createResponse.Data))
        : createResponse.Errors.ToActionResult();
}
```

Figuur 8: Create training command, handler en validatie

- ▼ Application
 - > Dependencies
 - ▼ Commands
 - > Certificate
 - > CertificateType
 - > FileStorage
 - > Team
 - ▼ Training
 - ▼ Create
 - cs# CreateTrainingCommand.cs
 - cs# CreateTrainingHandler.cs
 - cs# CreateTrainingValidation.cs
 - > Delete
 - > Update
 - > TrainingCategory
 - > User
 - > Dtos
 - > Extensions
 - > Interfaces
 - > Mediatr
 - > Queries
 - > Responses

Figuur 9: Linq query om een training op te halen via een id, in de mediatr handler

```
public class GetTrainingByIdHandler : IRequestHandler<GetTrainingByIdQuery, Response<TrainingResponseDto>>
{
    private readonly IAppDbContext _context;

    [MauroL.AllPhi]
    public GetTrainingByIdHandler(IAppDbContext context)
    {
        _context = context;
    }

    [MauroL.AllPhi]
    public async Task<Response<TrainingResponseDto>> Handle(GetTrainingByIdQuery request,
        CancellationToken cancellationToken)
    {
        var id:Guid = request.Id;
        var training = await _context.Trainings // DbSet<Training>
            .Where(t:Training => t.Id == id) // IQueryable<Training>
            .Select(t:Training => new TrainingResponseDto(
                t.Id,
                t.Title,
                t.CompletedAt,
                t.Provider,
                t.Description,
                t.TrainingCategoryId,
                t.UserId
            )).FirstOrDefaultAsync(cancellationToken); // Task<TrainingResponseDto?>

        return training is not null
            ? Response<TrainingResponseDto>.Ok(training)
            : Response<TrainingResponseDto>.Fail([Error.NotFound("No training found!")]);
    }
}
```

Figuur 10: EF-interceptor voor het soft deleten van een record

```
public class SoftDeletableInterceptor : SaveChangesInterceptor
{
    [MauroL.AllPhi]
    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData,
        InterceptionResult<int> result)
    {
        HandleSoftDeletes(eventData.Context);
        return base.SavingChanges(eventData, result);
    }

    [MauroL.AllPhi]
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData,
        InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        HandleSoftDeletes(eventData.Context);
        return base.SavingChangesAsync(eventData, result, cancellationToken);
    }

    [2 usages] [MauroL.AllPhi]
    private static void HandleSoftDeletes(DbContext? context)
    {
        if (context is null) return;

        var entries:IEnumerable<EntityEntry> = context.ChangeTracker // ChangeTracker
            .Entries()
            .Where(e:EntityEntry => e is { State: EntityState.Deleted, Entity: ISoftDeletable });

        foreach (var entry in entries)
        {
            var entity = (ISoftDeletable)entry.Entity;
            var deleteResult = entity.Delete();
            if (!deleteResult.IsSuccess)
            {
                entry.State = EntityState.Unchanged;
                continue;
            }

            entry.State = EntityState.Unchanged;

            foreach (var reference in entry.References) reference.TargetEntry?.State = EntityState.Unchanged;

            entry.Property(nameof(ISoftDeletable.IsDeleted)).IsModified = true;
            entry.Property(nameof(ISoftDeletable.DeletedAt)).IsModified = true;
        }
    }
}
```

Figuur 11: Training repository en de create methode voor een training

```
public class TrainingRepository : ITrainingRepository
{
    private readonly IAppDbContext _context;

    MauroL AllPhi
    public TrainingRepository(
        IAppDbContext context
    )
    {
        _context = context;
    }

    0+1 usages MauroL AllPhi
    public async Task<Result<Training>> CreateAsync(TrainingCreateDto dto,
        CancellationToken cancellationToken)
    {
        var trainingCategory = await _context.TrainingCategories // DbSet<TrainingCategory>
            .IgnoreQueryFilters() // IQueryable<TrainingCategory>
            .FirstOrDefaultAsync(
                t:TrainingCategory => t.Id == dto.TrainingCategoryId,
                cancellationToken
            ); // Task<TrainingCategory?>
        if (trainingCategory is null) return Result<Training>.Fail([Error.NotFound("Training category not found!")]);

        var user = await _context.Users.FirstOrDefaultAsync(t:User => t.Id == dto.UserId, cancellationToken);
        if (user is null) return Result<Training>.Fail([Error.NotFound("User not found!")]);

        var createResult = Training.Create(
            dto.Title,
            dto.Provider,
            dto.Description,
            dto.CompletedAt,
            dto.TrainingCategoryId,
            dto.UserId
        );
        if (!createResult.IsSuccess) return createResult;

        await _context.Trainings.AddAsync(createResult.Value, cancellationToken);
        await _context.SaveChangesAsync(cancellationToken);

        return Result<Training>.Ok(createResult.Value);
    }
}
```

Figuur 12: De interface die geïmplementeerd wordt door de training repository

```
public interface ITrainingRepository
{
    1 usage 1 implementation MauroL AllPhi
    Task<Result<Training>> CreateAsync(TrainingCreateDto dto, CancellationToken cancellationToken);
    1 usage 1 implementation MauroL AllPhi
    Task<Result> UpdateAsync(Guid id, TrainingUpdateDto dto, CancellationToken cancellationToken);
    1 usage 1 implementation MauroL AllPhi
    Task<Result> DeleteAsync(Guid id, CancellationToken cancellationToken);
}
```

Figuur 13: De Aspire AppHost die de projecten, database en blob storage start

```
using Microsoft.Data.SqlClient;
using Projects;

var builder = DistributedApplication.CreateBuilder(args);

var sqlServer :IResourceBuilder<SqlServerServerResource> = builder.AddSqlServer( name: "DefaultConnection")
    .WithLifetime(ContainerLifetime.Persistent)
    .WithDataVolume()
    .WithHostPort(54734);
var db :IResourceBuilder<SqlServerDatabaseResource> = sqlServer.AddDatabase( name: "TrainingCertificateTrackerDb");

db.WithCommand( name: "seed", displayName: "Seed Database", executeCommand: async (_) =>
{...});

var blobStorage :IResourceBuilder<AzureBlobStorageResource> = builder // IDistributedApplicationBuilder
    .AddAzureStorage( name: "Storage")
    .RunAsEmulator(azurite :IResourceBuilder<AzureStorageEmulatorResource> =>
    {
        azurite
            .WithLifetime(ContainerLifetime.Persistent)
            .WithDataVolume( name: "trainingcertificatetracker-blob");
    }) // IResourceBuilder<AzureStorageResource>
    .AddBlobs("blobs");

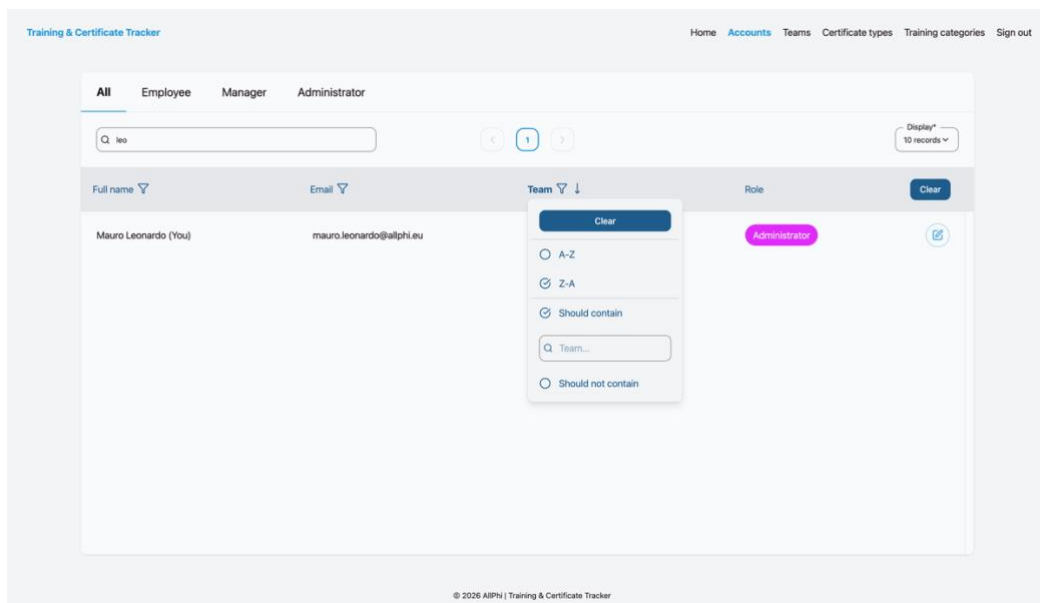
var orchestrator :IResourceBuilder<ProjectResource> = builder.AddProject<Aspire_Orchestrator>( name: "Orchestrator")
    .WithReference(db)
    .WaitFor(db);

var api :IResourceBuilder<ProjectResource> = builder.AddProject<API>( name: "ApiLayer")
    .WithReference(db)
    .WaitFor(db)
    .WithReference(blobStorage)
    .WaitFor(blobStorage)
    .WithReference(orchestrator)
    .WaitFor(orchestrator);

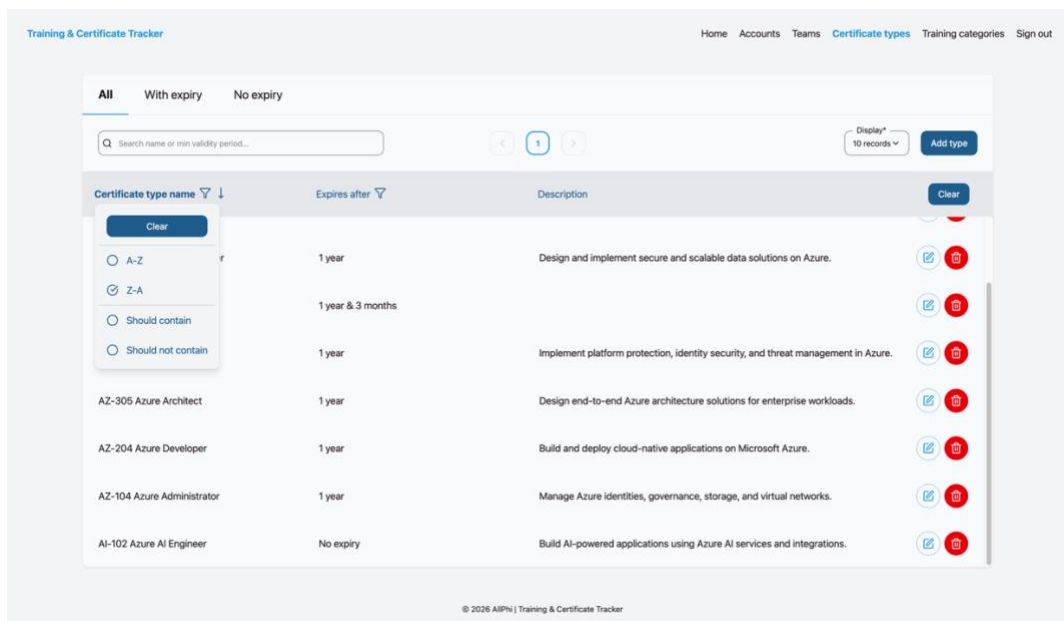
builder.AddJavaScriptApp( name: "Frontend", appDirectory: "../Frontend", runScriptName: "start")
    .WithReference(api)
    .WaitFor(api)
    .WithEnvironment( name: "VITE_API_URL", api.GetEndpoint( name: "https"))
    .WithHttpEndpoint(port: 4200, env: "PORT");

await builder.Build().RunAsync();
```

Figuur 14: Accountmanagement met de herbruikbare dropdown component open



Figuur 15: Certificaat type management met de herbruikbare dropdown component open



Figuur 16: De API-service voor certificaten die de API-calls bevat

```
export class CertificateApiService extends BaseApiService {
  private readonly endpoint: string = `${this.baseUrl}/certificates`;
  private readonly byTeamEndpoint: string = `${this.endpoint}/team`;
  private readonly byUserEndpoint: string = `${this.endpoint}/user`;

  public getStatistics(): Observable<ApiResponse<CertificateStatistics>> {...}

  public getStatisticsByTeamId(teamId: string): Observable<ApiResponse<CertificateStatistics>> {...}

  public getStatisticsByUserId(userId: string): Observable<ApiResponse<CertificateStatistics>> {...}

  public getById(certificateId: string): Observable<ApiResponse<Certificate>> {...}

  public getByTeamId( Show usages  MauroL-AllPhi +1
    teamId: string,
    selectedStatus?: 'All' | ExpiryStatus,
    search?: string,
    sorting?: string,
    skip?: number,
    take?: number,
  ): Observable<ApiResponse<Certificate[]>> {...}

  public getByUserId( Show usages  MauroL-AllPhi +1
    userId: string,
    selectedStatus?: 'All' | ExpiryStatus,
    search?: string,
    sorting?: string,
    includeDetails?: boolean,
  ): Observable<ApiResponse<Certificate[]>> {...}

  public create(certificateCreate: CertificateCreate): Observable<ApiResponse<Certificate>> {...}

  public update( Show usages  MauroL-AllPhi
    userId: string,
    certificateUpdate: CertificateUpdate,
  ): Observable<ApiResponse<void>> {...}

  public delete(userId: string): Observable<ApiResponse<void>> {...}
}
```

Figuur 17: De facade voor certificaten die de API-service gebruikt, met extra functionaliteiten zoals loading

```
export class CertificateFacade implements ResettableFacade {
  reset(): void { Show usages  ⚙ MauroL-AiPhi
    this.certificates = createModelState<Certificate[]>([]);
    this.certificateDetails.set( value: null);
  }

  private readonly loadingService: LoadingService = inject(LoadingService);
  private readonly toastService: ToastService = inject(ToastService);
  private readonly certificateService: CertificateApiService = inject(CertificateApiService);
  private readonly fileStorageFacade: FileStorageFacade = inject(FileStorageFacade);

  public certificates: ModelState<Certificate[]> = createModelState<Certificate[]>([]);
  public readonly certificateDetails : WritableSignal<Certificate | null> = signal<Certificate | null>( initialValue: null);

  public readonly certificateStatistics : WritableSignal<CertificateStatistics | nu... = signal<CertificateStatistics | null>( initialValue: null);

  public loadStatistics(): void {...}

  public loadStatisticsByTeamId(teamId: string): void {...}

  public loadStatisticsByUserId(userId: string): void {...}

  public loadCertificatesByTeamId( Show usages  ⚙ MauroL-AiPhi +1
    teamId: string,
    selectedStatus?: 'All' | ExpiryStatus,
    search?: string,
    sorting?: string,
    skip?: number,
    take?: number,
  ): void {...}

  public loadCertificatesByUserId( Show usages  ⚙ MauroL-AiPhi +1
    userId: string,
    selectedStatus?: 'All' | ExpiryStatus,
    search?: string,
    sorting?: string,
    includeDetails?: boolean,
  ): void {...}

  public async setCertificateDetails(certificateId: string): Promise<FacadeResponse> {...}

  public async createCertificate( Show usages  ⚙ MauroL-AiPhi
    certificateCreate: CertificateCreate,
    certificateCreateFile: File,
  ): Promise<FacadeResponse> {...}

  public async updateCertificate( Show usages  ⚙ MauroL-AiPhi
    certificateId: string,
    certificateUpdate: CertificateUpdate,
    certificateUpdateFile?: File,
  ): Promise<FacadeResponse> {...}

  public async deleteCertificate(certificateId: string): Promise<FacadeResponse> {...}
}
```

Figuur 18: Frontend modellen voor certificaten als interfaces

```
export interface Certificate { Show usages  ⚙ MauroL-AiPhi +2
  id: string;
  obtainedAt?: Date;
  expiresAt?: Date;
  expiryStatus?: ExpiryStatus;
  referenceNumber?: string;
  certificateTypeId: string;
  userName?: string;
  certificateTypeName?: string;
  userId?: string;
}

export interface CertificateCreate { Show usages  ⚙ MauroL-AiPhi
  obtainedAt: Date;
  overrideExpiresAt: boolean;
  expiresAt?: Date;
  referenceNumber: string;
  certificateTypeId: string;
  userId: string;
}

export interface CertificateUpdate { Show usages  ⚙ MauroL-AiPhi +1
  id: string;
  obtainedAt: Date;
  overrideExpiresAt: boolean;
  expiresAt?: Date;
  referenceNumber: string;
  certificateTypeId: string;
  userId: string;
}

export interface CertificateStatistics { Show usages  ⚙ MauroL-AiPhi
  totalCountOfActiveCertificates: number;
  totalCountOfExpiringCertificates: number;
  totalCountOfExpiredCertificates: number;
}
```

Figuur 19: Accountmanagementpagina als data table

Training & Certificate Tracker

Home Accounts Teams Certificate types Training categories Sign out

All Employee Manager Administrator

Q Search name, email, team...

1 2 3 >

Display 10 records

Full name	Email	Team	Role	
alice administrator	alice.admin@example.com	Regional Unit Brussel	Administrator	
anne an	anne.an@example.com	Regional Unit Gent	Employee	
bas bom	bas.bom@example.com	Regional Unit Westerlo	Employee	
bob administrator	bob.admin@example.com	Regional Unit Westerlo	Administrator	
daan daan	daan.daan@example.com	Regional Unit Westerlo	Employee	
elsa el	elsa.el@example.com	Regional Unit Westerlo	Employee	
fay fu	fay.fu@example.com	Regional Unit Gent	Employee	

© 2026 AllPh | Training & Certificate Tracker

Figuur 20: Certificate portfolio pagina als data table, met update venster open

Training & Certificate Tracker

Trainings Certificates

All Active Expiring Expired

Q Search reference number, type...

Reference number	Certificate type
CERT-LOTT-4-4368F684	AZ-104 Azure Administrator
CERT-LOTT-1-251CBD9D	SC-900 Security
CERT-LOTT-2-987746B4	AZ-305 Azure Architect
CERT-LOTT-3-75B033E2	DP-203 Azure Data Engineer

Update certificate 'CERT-LOTT-1-251CBD9D'

Reference number*
CERT-LOTT-1-251CBD9D

Category*
SC-900 Security

Obtained at*
18/06/2025

Expiry Enabled

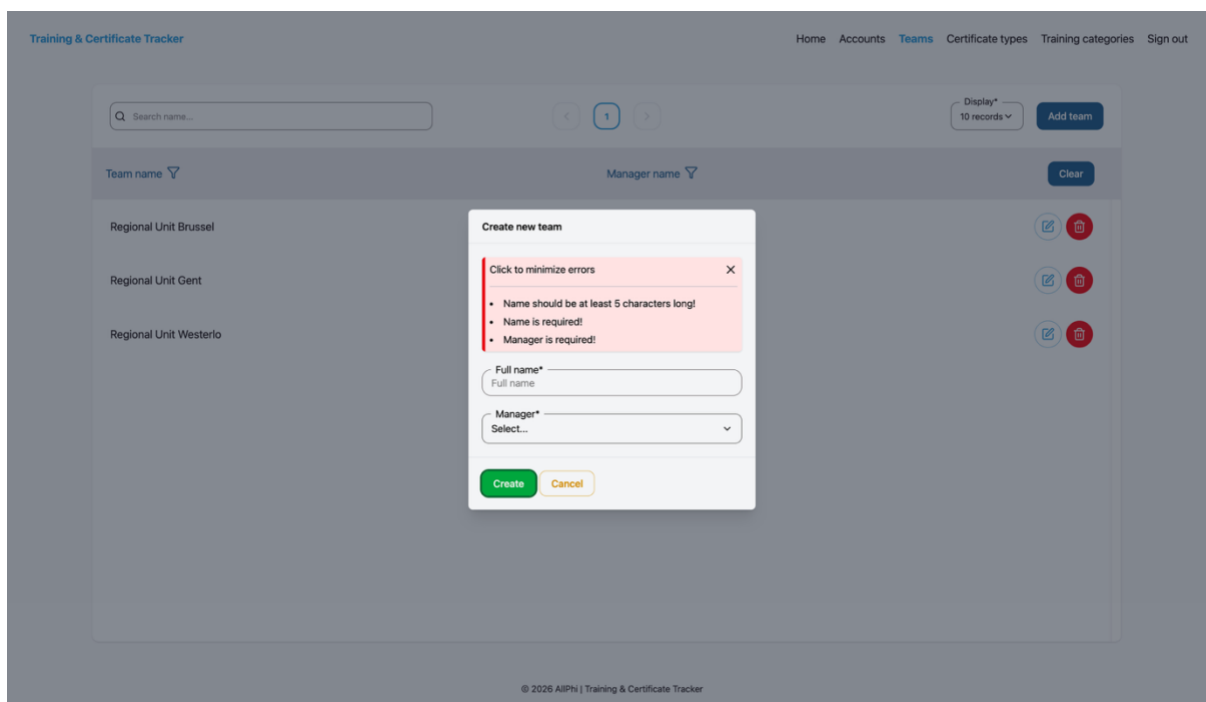
Override expiry date
17/06/2026

Choose a file to add or to replace the existing one.
Attachment
Choose a file...

Update Cancel

© 2026 AllPh | Training & Certificate Tracker

Figuur 21: Teammanagementpagina als data table, met create venster open met validatiefoutenvenster



Figuur 22: team skill matrix met muis over een record

Training & Certificate Tracker

Home Dashboard **Team Skills** Expiry Alerts Trainings Certificates

Reset	AZ-104 Azure Administrator	AZ-204 Azure Developer	AZ-305 Azure Architect	AZ-500 Azure Security	Certificate Management	DP-203 Azure Data Engineer	PL-300 Power
bas bom	-	⚠	-	✓	-	-	✓
daan daan	-	-	✓	✓	-	⚠	-
elsa el	⚠	✓	-	-	-	✓	-
frits fles	⏸	-	-	-	-	✓	⚠
koen koek	-	-	⚠	✓	-	✓	-
lars lucht	-	✓	-	⏸ lars lucht Expired	-	-	✓
linda manager	-	-	-	-	-	-	-
lotte lol	⏸	-	✓	-	-	✓	-
marit mop	-	-	✓	⚠	-	⏸	-

© 2026 AllPh | Training & Certificate Tracker

Figuur 23: team skill matrix met muis over een gebruikersnaam

Training & Certificate Tracker

Home Dashboard **Team Skills** Expiry Alerts Trainings Certificates

Reset	AZ-104 Azure Administrator	AZ-204 Azure Developer	AZ-305 Azure Architect	AZ-500 Azure Security	Certificate Management	DP-203 Azure Data Engineer	PL-300 Power
bas bom	-	⚠	-	✓	-	-	✓
daan daan	-	-	✓	✓	-	⚠	-
elsa el	⚠	✓	-	-	-	✓	-
frits flies	⏰	-	-	-	-	✓	⚠
koen koek	-	-	⚠	✓	-	✓	-
lars lucht	-	✓	-	⏰	-	-	✓
linda manager	-	-	-	-	-	-	-
lotte lol	⏰	-	✓	-	-	✓	-
marit mop	-	-	✓	⚠	-	⏰	-

© 2026 AllPh | Training & Certificate Tracker

Figuur 24: Dashboard pagina als administrator

